

Machine Learning Essentials – Exercise 1 Step-by-Step Guide

This guide explains how to approach and implement every task in Exercise 1. The assignment focuses on handwritten digit classification using: Nearest Mean Classifier Linear Discriminant Analysis (LDA) Support Vector Machines (SVM)
Dataset used: sklearn digits dataset (only digits 3 and 9).

1. Environment Setup

Step 1: Create a virtual environment.
Use either conda or uv.

Example with conda:
`conda create --name ml_homework python`
`conda activate ml_homework`
`conda install scikit-learn matplotlib jupyterlab`

Step 2: Start JupyterLab:
`jupyter lab`

Step 3: Create a notebook:
`linear-classifiers.ipynb`

2. Load and Explore the Dataset

Import libraries:

```
from sklearn.datasets import load_digits
import matplotlib.pyplot as plt
import numpy as np
```

Load the dataset:

```
digits = load_digits()
```

Extract:

- data = flattened 64-dimensional vectors
- images = 8x8 images
- target = labels
- target_names = class names

Check image size:

```
images.shape
```

You should discover:

1797 images of size 8x8.

Display one image of digit 3:

- Find an index where target == 3
- Plot using `plt.imshow()`

Try:

```
interpolation='nearest'
and
interpolation='bicubic'
```

3. Filter Digits 3 and 9

Create a binary classification dataset.

Keep only:

- digit 3
- digit 9

```

Convert labels:
3 -> -1
9 -> 1

Use boolean masks:
mask = (target == 3) | (target == 9)

Then split into train and test sets:

from sklearn.model_selection import train_test_split

Use:
test_size=0.4
random_state=0

```

4. Build a 2D Feature Function

Goal:
Reduce the original 64-dimensional data into 2 features.

Create a function:

```
def features2d(x):
    ...
```

Possible strategies:

1. Use two informative pixels
2. Difference between pixel groups
3. Sum of upper pixels vs lower pixels
4. Average brightness of different regions

Tip:
Visualize average images for 3 and 9.
Pixels with large differences are useful features.

5. Create a Scatter Plot

Use matplotlib scatter plots to visualize the two classes.

Plot:

- x-axis = feature 1
- y-axis = feature 2

Use different:

- colors
- markers
- sizes

Goal:
Check whether the classes separate well in 2D.

6. Implement the Nearest Mean Classifier

Compute:

- Mean feature vector for class -1
- Mean feature vector for class 1

Prediction rule:
Assign each sample to the nearest mean.

Steps:

1. Compute Euclidean distances
2. Choose the smaller distance
3. Return predicted label

Evaluate:

- training error
- test error

Error formula:

(number of wrong predictions) / (total samples)

7. Visualize Decision Regions

Create a 200x200 grid in feature space.

For each grid point:

- predict the class
- color the region

Overlay:

- training/test points
- class means
- decision boundary

Useful functions:

```
numpy.meshgrid()  
plt.contour()  
plt.imshow()
```

8. Implement LDA Training

Create:

```
def fit_lda(training_features, training_labels):
```

Compute:

1. Class means
2. Shared covariance matrix
3. Priors

Important:

Your implementation must work for:

- 2D features
- full 64D pixel vectors

Remove dead pixels:

- Compute variance for every pixel
- Remove pixels with variance < 0.001

9. Implement LDA Prediction

Create:

```
def predict_lda(mu, covmat, p, test_features):
```

Use:

```
beta = inv(covmat) @ (mu1 - mu_minus1)  
b = ...
```

Prediction:

```
sign(x @ beta + b)
```

Evaluate:

- training error
- test error

10. Visualize LDA

Visualize:

1. Decision regions
2. Scatter plot
3. Gaussian contours
4. Principal axes

Use:

```
numpy.linalg.eig()
```

Interpretation:

- Eigenvectors = directions
- $\sqrt{\text{eigenvalues}}$ = standard deviations

11. 10-Fold Cross Validation

Goal:

Estimate generalization performance.

Use:

```
from sklearn.model_selection import KFold
```

Steps:

1. Split data into 10 folds
2. Train on 9 folds
3. Test on 1 fold
4. Repeat 10 times
5. Average the errors

Compare:

Your implementation vs sklearn LDA.

12. Implement SVM Training

Create:

```
def fit_svm(...):
```

Implement gradient descent.

Loss:

```
0.5 * ||beta||^2 + lambda * hinge_loss
```

Key steps:

1. Initialize beta randomly
2. Compute gradients
3. Update beta and b
4. Repeat for many iterations

Use learning rate scheduling:

If training stagnates:

```
learning_rate /= 10
```

13. Implement SVM Prediction

Prediction rule:

```
sign(x @ beta + b)
```

Evaluate:

- training error
- test error

Compare:

- nearest mean
- LDA
- SVM

14. Visualize SVM Training

Create plots for:

1. Loss vs iteration
2. Training error vs iteration

This helps analyze:

- convergence
- stability
- learning behavior

15. Final Report Checklist

Before submission make sure you have:

- ✓ linear-classifiers.ipynb
- ✓ linear-classifiers.html
- ✓ Clear explanations
- ✓ Scatter plots
- ✓ Decision region plots
- ✓ LDA visualizations
- ✓ SVM visualizations
- ✓ Cross-validation results
- ✓ Error rates
- ✓ Comparison with sklearn implementations

Finally:
Zip everything into:
ex01.zip

Recommended Workflow: Finish the data exploration first Implement and debug features2d() Build nearest mean classifier Implement LDA Implement SVM last Add visualizations throughout the notebook Good luck with your assignment!